

StockVisible

See the demand that stock-outs hide, then decide what to reorder within budget — with forecasts whose reliability is proven.

Dossier date	27 September 2026
Live demo	https://stockvisible.galsentechnologie.com
Fallback	https://stockvisible.onrender.com
Repository	https://github.com/Mouhinf/stockvisible

In short

- 20.2% of the hours from 6:00 to 22:00 are out of stock in the training data: observed sales underestimate demand.
- Three screens: Verify (the evidence), Understand (one product's demand), Buy (basket within budget, validated then exported).
- ML engine chosen on validation only, frozen, then evaluated a single time: MAE 3.2% lower than B1 on the final test, with a negative bias shown on screen.
- Real FreshRetailNet-50K data (CC BY 4.0), normalised: no currency unit, no data from Senegal.

Every figure in this dossier comes from README.md, logs/final_test_result.json, docs/judging-evidence.md or AI_USAGE.md (repository, branch final-hackathon-polish). Sales values are normalised, with no unit.

1. Problem and user

When a product is out of stock, the till records zero sales. That zero is not an absence of customers: observed sales underestimate demand, and a manager who reorders on that basis reorders too little, which sustains the stock-outs.

In the data used (500 real store × product series of fresh produce), 20.2% of the hours from 6:00 to 22:00 are out of stock over the training period.

Target user: the manager or purchasing lead of a fresh-produce outlet, who must decide every day what to reorder with a limited budget. No interview or pilot with a manager has taken place yet: this need is a working assumption, not a need validated in the field.

2. What the application does

The manager does not need to read the code: the application first shows them the evidence of reliability (Verify screen), then the demand for one product (Understand), then a basket within budget that they validate and export (Buy).

Screen	What you see
Verify (home)	The problem in one sentence, then the evidence: errors of the ML engine and of the two baselines on validation and on the final test, uncertainty, guarantees, limitations
Understand	For one product: hour-by-hour sales, availability, observed sales vs estimated demand per day, B0 / B1 forecasts, abstention; import of a file checked against the data contract
Buy	Costs, lots and stock levels entered as assumptions; budget; optimal basket (covered demand, shortfall); validation then CSV / JSON export with provenance

3. Data

- FreshRetailNet-50K, Dingdong-Inc, CC BY 4.0 licence, revision 08c1fab7f9257bc73679d415d65d644165d351d4 (arXiv:2505.16319).
- Deterministic subset of 500 series (seed 42), filtered without modifying any value. Sales normalised by the provider: no currency unit.
- Real / simulated separation: the application's reference data is this real benchmark. No synthetic FCFA scenario was built; costs, lot sizes and stock levels are assumptions entered by the user. The only synthetic files are the test fixtures, named *_SYNTHETIQUE.csv.
- The dataset's official "eval" file is not versioned and has never been used to evaluate a model.

4. Method and result

B0: median of sales in the same time slot, stock-outs included. B1: the same median restricted to hours declared available, with abstention. ML: quantile gradient boosting (scikit-learn) on strictly past features.

Rolling day-ahead (J+1) hourly forecasts, hours when the product was available, the same hours for all three models. On the final test, opened a single time, the ML's error is 3.2% lower than B1's (5.4% on validation). It under-forecasts (negative bias, close to B0): this is shown on screen, and it is why the basket does not use it.

The masked-data test is a reconstruction / fidelity test: it measures the fidelity of the reconstruction, not the real lost demand. It is not proof of recovering real demand.

Model	Validation MAE	Final test MAE	Final test bias
ML (frozen engine)	0.0456	0.0492	-0.0355
B1 (median, available hours)	0.0482	0.0508	-0.0267
B0 (median, stock-outs included)	0.0475	0.0508	-0.0358

```
{
  "ouvertures_du_test": 1,
  "horodatage_début_utc": "2026-09-26T22:21:13+00:00",
  "horodatage_fin_utc": "2026-09-26T22:23:00+00:00",
  "moteur_gelé": "ML",
  "résultat_moteur_test": {"mae": 0.04915226966771911, "bias": -0.03552760990267266, "couverture": 1.0, "n": 134744},
  "contexte_test": {
    "B0 (glissant)": {"mae": 0.0507987368639791, "bias": -0.03578285489520869, "couverture": 1.0, "n": 134744},
    "B1 (glissant)": {"mae": 0.05077402333313543, "bias": -0.026656808466425222, "couverture": 0.9807611111111111, "n": 134744}
  }
}
```

5. Scientific discipline

DATA → TRAIN (60 d) → VALIDATION (15 d) → ENGINE CHOICE → FREEZE → FINAL TEST ONCE ONLY (15 d)

- Chronological split per series, never random.
- Selection rule written before any result: the ML is kept only if it beats B1 on validation, otherwise B1 becomes the engine.
- Anti-leakage tested: canaries, perturbation of future days, and 9 negative controls (leaks injected on purpose, which the suite must detect).
- Two looks at the validation set are declared: the first model was degenerate (it predicted 0 everywhere); it was fixed once, then the configuration was frozen.

6. Architecture

Function	Modules	Role
F1 DATA	data.py, validation.py	Deterministic, verified (SHA-256) subset of the dataset; data contract; safe reading of imported files (CSV / Parquet, size and row caps, no execution of content)
F2 REVEAL	ui/pages/comprendre.py, ui/charts.py	Hourly sales, declared availability, estimated demand
F3 VERIFY	splits.py, baselines.py, features.py, model.py, evaluation.py, selection.py, final_test.py, uncertainty.py	Chronological split, B0 / B1 baselines, strictly past features, ML model, selection on validation, freeze, single final test, empirical interval
F4 DECIDE	allocation.py	Optimal basket within budget, bounded exhaustive enumeration (≤ 3 products), whole lots
F5 ACT	exports.py	Basket validation, CSV / JSON export strictly identical to the screen

app.py: router of the three screens (st.navigation). No model is trained at run time: the Verify screen reads engine_freeze.json, logs/final_test_result.json and logs/interval_calibration.json.

Interface: Streamlit 1.64, Plotly charts, design system in design-system/MASTER.md. Data contract and decision log: docs/data-contract.md.

7. Responsible AI

- No LLM and no external AI API in the application: the engine is a tabular model competing against two simple rules.
- Visible abstention: when no comparable period exists, B1 predicts nothing and the screen says so; the hours stay empty, never filled by default.

- Measured uncertainty: 80% interval around the engine, empirical coverage of 79.5% on validation days never seen during calibration; “interval not calibrated” is shown when support is insufficient. At night the interval is [0, 0]: calibrated but not very informative.
- Real / simulated separation: real data flagged as such; costs and stock levels flagged as assumptions; synthetic test files labelled; lost demand always presented as estimated. Limitations shown on screen.

8. Limitations and unverified assumptions

- Real Chinese data (Dingdong), normalised: no currency unit, no FCFA, no data from Senegal.
- Costs, lot sizes and stock levels are entered assumptions: the dataset contains none.
- Demand lost during a stock-out is estimated, never observed. The masked-data test measures the fidelity of the reconstruction, not the real lost demand.
- Modest ML gain (3.2% on the test) and negative bias; the basket uses B1.
- The rows of the test period were read before the split (data quality check) and are public in the repository; they were not used for any model decision.
- Render free tier: cold start observed at about 2 minutes after 15 minutes of inactivity.
- No interview or pilot with a manager: the user need remains a working assumption.

9. Judging criterion → evidence

Criterion (weight)	Evidence	Test
Problem and value (20)	20.2% of 6:00–22:00 hours out of stock; problem banner computed from the data; observed sales and estimated demand side by side	<code>test_problem_banner_numbers_are_computed_not_typed</code>
Functionality (20)	Import → visualisation → evidence → budget → basket → validation → export path, local and production, desktop and mobile, zero console errors; export identical to the screen	<code>e2e/parcours.spec.ts ; test_export_is_strictly_what_the_screen_shows</code>
Use of AI (20)	ML against B0 / B1, rule written before any result, choice on validation only, frozen configuration (SHA-256 fingerprints); final test opened a single time	<code>test_select_engine_never_receives_test_data ; test_result_is_a_single_opening_of_the_frozen_engine</code>
Testing and reliability (15)	Reproducible split; anti-leakage through canaries and perturbation; 9 negative controls; dress rehearsal of the final test; GitHub Actions CI	<code>test_anti_leak_fails_on_injected_leak ; test_dress_rehearsal_on_validation_reproduces_frozen_metrics</code>
Experience and demo (15)	Three screens Verify → Understand → Buy; at most three metrics per screen; computed WCAG AA contrasts	<code>test_at_most_three_metrics_per_screen ; test_every_text_color_meets_wcag_aa</code>
Responsible AI and data (10)	Visible abstention; interval coverage 79.5%; real CC BY 4.0 data, pinned revision; assumptions and synthetic files labelled	<code>test_synthetic_series_without_enough_history_triggers_abstention ; test_real_report_uses_validation_only_and_is_consistent</code>

10. Installation, tests and deployment

Installation (Python 3.12 required)

```
python3.12 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/streamlit run app.py # http://localhost:8501
```

```
.venv/bin/ruff check .
.venv/bin/pytest -q # full suite (519 tests collected)
.venv/bin/pytest -q -m "not touches_test_period" # without reading the reserved test period
npm ci && npx playwright test # E2E path, desktop + mobile
E2E_BASE_URL=https://stockvisible.onrender.com npx playwright test # same path in production
```

On a fresh clone, 2 tests are skipped because they need data files that are not versioned (official eval, download cache), with an explicit reason. GitHub Actions CI: ruff, pytest, pip-audit (informational). Free Render instance (512 MB, sleeps after 15 min of inactivity). No secret is needed.

Render deployment (Frankfurt region, redeployed on every push to master)

Field	Value
Runtime	Python 3
Branch	master
Build Command	pip install -r requirements.txt
Start Command	streamlit run app.py --server.port \$PORT --server.address 0.0.0.0 --server.headless true
Health Check Path	/_stcore/health
PYTHON_VERSION	3.12.3

11. AI and tools declaration

- In the product: a scikit-learn tabular model, with no LLM and no external AI API. NVIDIA Brev: not used.
- To build it: Claude Code (Anthropic) as programming assistant — Claude Sonnet 4.6 for the first commit, Claude Opus 5.5 afterwards; a separate Claude session for the 4 finishing commits of pull request #1; read-only Claude agents for the security audit and a simulated evaluation against the jury grid. Skills: stockvisible-data-contract and dataviz. No MCP server.
- The project owner sets the rules and takes the decisions traced in the Git history; every commit carries the corresponding Co-Authored-By line.
- Timeline (UTC, Git history): M0 → M10 on 26/09/2026 from 16:03 to 23:29; M11 → M17 on 27/09/2026 from 00:56 to 13:11. The rules on code prepared before the event are those of the official regulations.

12. Licences

- Data: FreshRetailNet-50K, Dingdong-Inc, CC BY 4.0.
- Dependencies: open-source libraries listed in requirements.txt (pandas, NumPy, scikit-learn, Streamlit, Plotly, pyarrow, pytest, ruff, pip-audit) and package.json (@playwright/test, testing tool only), each under its own licence.
- StockVisible code: no licence file has been added to the repository yet; without an explicit licence, rights remain reserved to the author.
- Tab logo: Galsen Technologie icon (galsentechnologie.com).

13. After the hackathon

- Pilot with an outlet equipped with a till and stock tracking, with its real costs.
- Then a purchase-oriented engine (quantile above the median) chosen on validation and evaluated on the dataset's official "eval" period, never used to evaluate a model.
- Choose a licence for the code: no licence file has been added to the repository yet.

Timeline

The Git history is authoritative. First commit on 26/09/2026 at 16:03 UTC; data, model, freeze, final test and screens (M0 to M11) committed on 26/09/2026 and overnight until 27/09 at 00:56 UTC; E2E tests, security audit, export, CI, production and finishing work on 27/09/2026.

14. Demo and jury audit

- Demo script and video (M18): to come.
- Versioned jury audit (M19): to come.